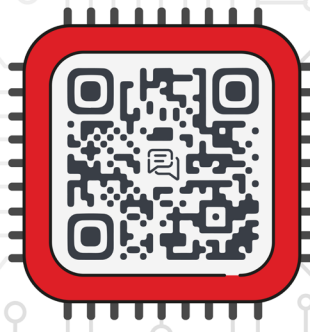




نظري

1



6



900 sp

كلية الهندسة المعلوماتية

السنة الأولى

مقدمة في البرمجة كائنية التوجه

د. فراس ضعيف

د. معتر النجرس



البرمجة 2

RB Informatics; 29/03/2026

تبنى موضوعات المقرر «البرمجة 2» على المفاهيم الأساسية التي تمت دراستها في المقرر «البرمجة 1»، ولذلك تعد الإحاطة بهذه المفاهيم شرطاً أساسياً لفهم محتوى المقرر الجديد. ومن أهم هذه المتطلبات:

- البنى الشرطية (Conditionals): مثل if-else و switch.
- الحلقات التكرارية (Loops): مثل for و while و do-while.
- المصفوفات (Arrays): سواء كانت أحادية البعد أو ثنائية البعد.

التحول في نمط البرمجة

يمثل هذا المقرر انتقالاً من أسلوب البرمجة الإجرائية (Procedural Programming) الذي تم اعتماده في البرمجة 1، حيث كان التركيز منصباً على حل المسائل من خلال مجموعة من الدوال المستقلة، إلى أسلوب البرمجة كائنية التوجه (Object-Oriented Programming - OOP).

في هذا النمط تعامل مكونات النظام المختلفة على أنها كائنات (Objects)، يمتلك كل منها خصائص وسلوكيات محددة. وتبنى هذه السلوكيات باستخدام الأساسيات البرمجية التي تمت دراستها سابقاً، إلا أن التركيز يصبح موجهاً بدرجة أكبر نحو تصميم النظام ونمذجته (Modeling) وتنظيم مكوناته، بدلاً من الاقتصار على كتابة تعليمات برمجية منفصلة لحل مسائل جزئية.

دواعي استخدام البرمجة كائنية التوجه (Why OOP)

تظهر أهمية البرمجة كائنية التوجه عند الانتقال من البرامج البسيطة التي تنفذ عمليات حسابية محدودة، إلى الأنظمة الواقعية الكبيرة مثل المتاجر الإلكترونية والمكتبات والأنظمة الجامعية.

التحديات في البرمجة الإجرائية:

1. محدودية أنواع البيانات:

تعتمد البرمجة الإجرائية بصورة أساسية على أنواع البيانات الأولية مثل int و bool و char، في حين تحتاج الأنظمة الكبيرة إلى تمثيل كيانات واقعية أكثر تعقيداً، مثل الطالب والأستاذ والمادة.

2. تشتت البيانات وتكرارها:

عند التعامل مع بيانات عدد كبير من الطلاب، كأن يصل العدد إلى 10,000 طالب مثلاً، يصبح من الضروري تعريف متغيرات متعددة لكل طالب (الاسم، الرقم، العمر، تاريخ الميلاد وغيرها)، وهو ما يؤدي إلى تضخم حجم البرنامج وصعوبة إدارته.

3. غياب الترابط بين البيانات:

تكون البيانات موزعة في متغيرات مستقلة، مما يجعل تمريرها بين الدوال أكثر تعقيداً، ويرفع احتمالية الوقوع في الأخطاء عند تعديل البرنامج أو تطويره.

الحل الذي تقدمه البرمجة كائنية التوجه:

يمكن النظر إلى البرمجة كائنية التوجه بوصفها امتداداً أكثر تطوراً لمفهوم السجلات (Structures) المعروف في البرمجة 1، إذ تتيح تجميع البيانات المرتبطة مع السلوكيات الخاصة بها ضمن وحدة واحدة متماسكة. ويساعد ذلك على الحفاظ على ترابط البيانات، وتسهيل عمليات الصيانة والتعديل، وتقليل التأثيرات الجانبية الناتجة عن تطوير أجزاء من النظام.

كما تفرض هذه المنهجية التفكير في الكيانات المكونة للنظام وكيفية تمثيلها على شكل كائنات، مع تحديد ما يجب أن يكون متاحاً للاستخدام الخارجي (Public) وما ينبغي إخفاؤه أو تقييد الوصول إليه (Private).

المفاهيم الأساسية للبرمجة كائنية التوجه

أولاً: الصنف (Class)

الصنف هو فئة أو قالب يجمع مجموعة من الكائنات التي تشترك في خصائص وسلوكيات معينة. ويمكن تعريفه على أنه مخطط أو نموذج مجرد (Template/Blueprint) يصف السمات المشتركة لمجموعة من الكائنات.

أمثلة توضيحية:

صنف الكائنات الحية: يمتلك خصائص مثل الكتلة، وسلوكيات مثل تبادل المادة والطاقة والتكاثر. ومن الناحية البرمجية تمثل الخصائص متغيرات (Variables)، بينما تمثل السلوكيات توابع أو دوال (Methods).
صنف الطلاب: يُعد أكثر تخصصاً من صنف الكائنات الحية، إذ يضيف خصائص مثل العمر، وسلوكيات مثل الدراسة وتقديم الامتحانات.

أنواع الخصائص (Attributes / Properties)

تنقسم الخصائص المعرفة داخل الصنف إلى نوعين رئيسيين:

1. الأنواع الأولية (Primitive Types):

وهي الأنواع الأساسية المعرفة ضمن اللغة مثل int و double و boolean.

2. الأنواع الصنفية (Class Types):

وهي الخصائص التي يكون نوعها عبارة عن صنف آخر سبق تعريفه، مثل وجود خاصية من نوع «مادة» داخل صنف «طالب».

```
class Subject {
    String subjectName;
    int mark;
}

class Student {
    int ID; // Primitive type
    String studentName;
    Subject studentSubject = new Subject(); // Class type
}
```

ثانياً: التجريد (Abstraction)

يُقصد بالتجريد القدرة على النظر إلى الأشياء من خلال خصائصها العامة والأساسية، مع تجاهل التفاصيل الجزئية أو الفروقات الثانوية التي لا تؤثر في فهم الفكرة العامة. فمثلاً، عندما نسمع كلمة «سيارة»، فإن الذهن يستحضر مفهوماً عاماً للسيارة، بغض النظر عن نوعها أو لونها أو سنة تصنيعها. فقد تكون السيارة من طراز حديث، أو سيارة قديمة من خمسينيات القرن الماضي، ومع ذلك فإننا ندركها جميعاً بوصفها «سيارة» يرجع ذلك إلى أن العقل لا يركّز في هذه الحالة على التفاصيل الخاصة بكل نموذج، بل على السمات المشتركة بين جميع السيارات، مثل وجود العجلات، والمقود، والمكابح، وقدرتها على الحركة على الطرق. وبذلك يتم تمثيل السيارة كمفهوم عام يشمل أنواعاً متعددة وأشكالاً مختلفة.

ويمكن توضيح ذلك أيضاً من خلال مهارة قيادة السيارة؛ فالشخص الذي يتعلم قيادة سيارة معينة يستطيع غالباً قيادة سيارات أخرى، سواء كانت حديثة أو قديمة، لأن آلية القيادة الأساسية متشابهة. فهو لا يتعامل مع كل سيارة كحالة منفصلة تماماً، بل يعتمد على فهمه المجرد للمكونات والوظائف الأساسية للسيارة.

ومن هنا، فإن التجريد يساعدنا على تعميم المعرفة، وفهم الأشياء من خلال خصائصها الجوهرية المشتركة، بدلاً من الانشغال بالتفاصيل الفردية لكل حالة. وهذا ما يجعل التجريد مبدأً أساسياً في التفكير المنطقي، وفي بناء المفاهيم، وفي مجالات مثل البرمجة وتحليل الأنظمة.

ثالثاً: الكائن / الغرض (Object)

الكائن هو مثيل فعلي (Instance) يتم إنشاؤه اعتماداً على المخطط المحدد في الصنف (Class)، بحيث يمتلك الخصائص والسلوكيات التي تم تعريفها ضمن ذلك الصنف.

ملاحظة: عند تصميم الأنظمة البرمجية، يبدأ العمل عادةً بتحليل الكيانات والأغراض الموجودة في الواقع، ثم تستخلص منها الأصناف المناسبة، وليس العكس.



مثال برمجي متكامل (Class & Object)

```

1 class Subject {
2     String subjectName;
3     int mark;
4 }
5 class Student {
6     int ID;
7     String studentName;
8     Subject studentSubject;
9     boolean executedExam(boolean x) {
10         return x;
11     }
12 }
13 public class Main {
14     public static void main(String[] args) {
15         Student s1 = new Student();
16         Subject sub1 = new Subject();
17         sub1.mark = 100;
18         sub1.subjectName = "English";
19         s1.studentName = "Ahmed";
20         s1.studentSubject = sub1;
21         s1.ID = 154;
22         s1.executedExam(false);
23     }
24 }

```

السطر	الشرح
1,5	إنشاء صفوف جديدة
2,3,6,7,8	إضافة خصائص (Attributes) لكل صف
9	تعريف تابع (Method) للصف Student
15	إنشاء Object من الصف Student باسم s1
16	إنشاء Object من الصف Subject باسم sub1
17,18	إضافة قيم لخصائص الكائن English
19,20,21	إضافة قيم لخصائص الكائن s1

- سنتعرف بالتفصيل على مفاهيم Java المتعلقة بال OOP في المحاضرات القادمة
- سنتعرف على syntax لغة Java في محاضرات العملي ومحاضرات ال TD

ركائز البرمجة كائنية التوجه

ترتكز البرمجة كائنية التوجه على أربع خصائص أساسية تشكل جوهر هذا النمط البرمجي، وسيجري تناولها بالتفصيل في المحاضرات اللاحقة:

1. التغليف (Encapsulation):

ويقصد به حماية البيانات وتنظيم صلاحيات الوصول إلى الخصائص والميزات داخل الصنف، بحيث يُحدد ما يمكن الوصول إليه وما يجب تقييده.

2. الوراثة (Inheritance):

وتتمثل في إنشاء أصناف جديدة تستمد خصائصها وسلوكياتها من أصناف أكثر عمومية وتجريداً، مثل اشتقاق صنف «طالب» من صنف «إنسان». وتكون الأصناف المشتقة أكثر تخصصاً وأقل تجريداً من الأصناف الأصلية.

3. تعدد الأشكال (Polymorphism):

تعدد الأشكال يعني إمكانية استخدام التابع نفسه بأشكال تنفيذ مختلفة، تبعاً للكائن المستدعي أو لطريقة الاستدعاء، وينقسم إلى قسمين:

- Method Overriding يُحدّد التنفيذ المناسب أثناء وقت التشغيل Runtime Polymorphism.
- Method Overloading يُحدّد التنفيذ المناسب أثناء الترجمة Compile-Time Polymorphism.

4. التجريد (Abstraction):

ويهدف إلى إخفاء التفاصيل الداخلية المعقدة، وإظهار الواجهات الأساسية الضرورية فقط.

بيئة ولغة Java مقارنة بلغة C++

تتمتع لغة Java بميزة أساسية تميزها عن لغة C++ وهي الاستقلالية عن منصة التشغيل (Cross-platform)، والتي تُعرف بالشعار الشهير:

"Write once, run anywhere"

وقد ساهمت هذه الميزة في انتشار Java على نطاق واسع في التطبيقات المؤسسية والأنظمة الكبيرة، إذ يمكن تشغيل البرنامج نفسه على أنظمة تشغيل مختلفة دون الحاجة لإعادة كتابة الكود أو تعديله بما يتوافق مع كل منصة على حدة.

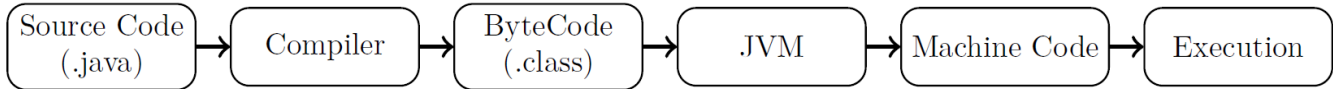
وجه المقارنة:

في لغة C++: تتم عملية تجميع (Compile) للكود المصدري لإنتاج ملف تنفيذي مثل exe. على نظام Windows. ويكون هذا الملف مرتبطاً بالمنصة التي جرى التجميع عليها، لذلك لا يمكن تشغيله مباشرة على أنظمة أخرى مثل Linux أو MacOS دون إعادة تجميع البرنامج.

في لغة Java: يُجمع البرنامج مرة واحدة لينتج ملفاً بسيطاً يُعرف بـ ByteCode، ويمكن تشغيل هذا الملف على أنظمة تشغيل مختلفة دون الحاجة إلى إعادة التجميع.

مراحل ترجمة وتنفيذ برنامج Java

تمر عملية تنفيذ برنامج Java بعدة مراحل متتالية تبدأ بكتابة الكود المصدري ضمن ملفات تحمل اللاحقة .java. بعد ذلك يُترجم هذا الكود باستخدام المترجم (java compiler) javac لينتج ملفات بلاحقة class. تحتوي على الـ ByteCode وعند تشغيل البرنامج تتولى الـ JVM تحميل هذا الـ ByteCode وتنفيذه على نظام التشغيل المستهدف.



آلية العمل وبيئة التشغيل في Java

تعتمد Java في تحقيق الاستقلالية عن منصة التشغيل على مجموعة من المكونات البرمجية المتكاملة، أهمها:

1. الآلة الافتراضية لجافا (JVM - Java Virtual Machine)

تمثل الـ JVM طبقة برمجية تقع بين برنامج Java ونظام التشغيل. وتوجد نسخة خاصة منها لكل نظام تشغيل، بحيث تتولى قراءة ملفات الـ ByteCode وتنفيذها، مما يعزل البرنامج عن التفاصيل الخاصة بالمنصة التي يعمل عليها.

2. مترجم الوقت الفعلي (JIT - Just-In-Time Compiler)

أضيف هذا المكوّن لتحسين الأداء وتقليل البطء الناتج عن التفسير المستمر للتعليمات. فبدلاً من الاكتفاء بقراءة الـ ByteCode وتنفيذه سطرًا بسطر، يقوم الـ JIT بمراقبة الأجزاء التي تنفذ بشكل متكرر داخل البرنامج، والتي تعرف غالباً باسم Hot Spots، ثم يترجمها مباشرة إلى لغة الآلة (Machine Code) ويحتفظ بالنتائج لاستخدامها لاحقاً. ونتيجة لذلك تنفذ هذه الأجزاء بسرعة أكبر عند تكرار استدعائها، مما يرفع كفاءة البرنامج أثناء التشغيل.

مكونات بيئة Java

تتكون بيئة Java من مكونات أساسية أهمها JVM و JRE و JDK، ولكل منها وظيفة مختلفة ضمن بيئة العمل.

المكوّن	الوظيفة
JVM (Java Virtual Machine)	تنفيذ الـ ByteCode والتعامل مع نظام التشغيل
JRE (Java Runtime Environment)	بيئة التشغيل التي تحتوي على الـ JVM والمكتبات اللازمة لتشغيل البرامج
JDK (Java Development Kit)	حزمة تتضمن أدوات البرمجة والترجمة بالإضافة إلى مكونات الـ JRE

وبذلك يمكن اعتبار الـ JDK البيئة المستخدمة لتطوير برامج Java وتجميعها، في حين تُستخدم مكونات الـ JRE والـ JVM بصورة أساسية أثناء تشغيل البرامج وتنفيذها.

